

St. Francis Institute of Technology**(An Autonomous Institution)**

AICTE Approved | Affiliated to University of Mumbai
A+ Grade by NAAC: CMPN, EXTC, INFT NBA Accredited: ISO 9001:2015 Certified

Department of Information Technology

A.Y. 2025-2026

Class: BE-IT A/B, Semester: VII

Subject: Secure Application Development Lab

Student Name: Tanmay Bhatkar

Student Roll No: 14

Experiment – 4: Study of different SAST (Static Application Security Testing) tools**Aim:** To study and exercise on different SAST (Static Application Security Testing) tools**Objective:** After performing the experiment, the students will be able to –

- To get familiar with the features provided by the open source SAST tools on GitHub

Lab objective mapped: ITL703.2: To understand the methodologies and standards for developing secure code**Prerequisite:** Basic knowledge Information Security, software engineering**Requirements:** Personal Computer, Windows operating system browser, Internet Connection etc.**Pre-Experiment Theory:****What is Static Application Security Testing (SAST)?**

SAST is a vulnerability scanning technique that focuses on source code, byte code, or assembly code. The scanner can run early in your CI pipeline or even as an IDE plugin while coding. SAST tools monitor your code, ensure protection from security issues such as saving a password in clear text or sending data over an unencrypted connection.

How do SAST tools work?

SAST is a technique used to evaluate source code without actually executing it. It involves examining the program's structure and syntax to identify potential issues and errors, such as coding mistakes, security vulnerabilities, and performance bottlenecks. The process involves parsing the source code, building an abstract syntax tree, and applying various analysis

techniques to detect issues. By providing early feedback on potential issues in the code, SAST can help improve software quality and reduce the likelihood of errors and security vulnerabilities.

01	Configuration analysis	- Checks the application configuration files. - Ensures that the configuration is aligned with security practices and policies, such as defining a default error page for the web application.
02	Semantic analysis	- Tokenization and examination of syntax, identifiers, and resolving types from code - SAST tools are able to analyze a particular code within its context, such as detecting SQL injections through <code>*.executeQuery()</code>.
03	Dataflow analysis	- Tracks the data flow through the application to determine if input is validated before use. - Determines whether data coming from insecure source such as a file, the network or user input is cleansed before consumption.
04	Control flow analysis	Checks the order of the program operations to detect potentially dangerous sequences such as race conditions, secure cookie transmission failure, uninitiated variables, or misconfigurations of utilities before use.
05	Structural analysis	- Examines language-specific code structures for inconsistencies with secure programming practices and techniques. - Identifies weaknesses in class design, declaration and use of variables and functions - Identifies issues with generation of cryptographic material and hardcoded passwords.

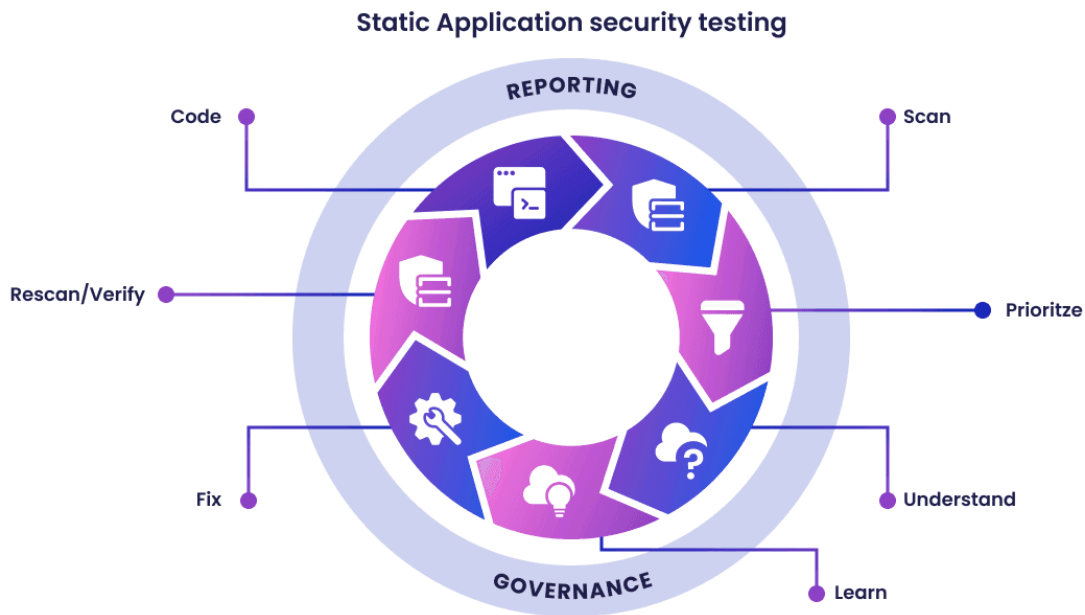
What vulnerabilities can SAST tools find?

SAST tools detect a range of security incidents and vulnerabilities in source code, including:

- Dataflow issues
- Semantic errors
- Misconfigured settings
- Control flow problems
- Structural flaws
- Memory issues

7 stages of a security application testing scan

The seven stages of a SAST scan are:



1. **Scan** your code as it's being written.
2. **Prioritize** and triage based on the severity and impact of the vulnerabilities found
3. **Understand** the nature of the vulnerabilities found by reviewing scan data and assessing the associated risk level.
4. **Learn** from the scan findings to prevent similar vulnerabilities in the future. This includes improving code quality, adopting secure coding practices, and implementing developer security training.
5. **Fix** vulnerabilities found in the scan by patching the code or implementing other remediation measures.
6. **Rescan** to verify that the fix has worked.
7. **Continue** coding while integrating security into the development process to prevent vulnerabilities from being introduced in future code.

Procedure:

Select any **five** open source SAST tools from GitHub and discuss them in detail.

Post-Experimental Exercise

Questions:

- How to find the right SAST tool to secure the software development lifecycle (SDLC)?
- Compare SAST with DAST

Conclusion:

- Write what was performed in the experiment.
- Write the significance of the topic studied in the experiment.

References

- <https://snyk.io/learn/application-security/static-application-security-testing/#how>
- <https://github.com/analysis-tools-dev/static-analysis>

1) Bandit — Python (AST-based security scanner)



Bandit is a focused security-oriented static analyzer for Python that parses source files into an AST and runs plugin checks to spot common issues (hard-coded creds, insecure subprocess usage, weak crypto, etc.). It was born in the OpenStack project and is now maintained by PyCQA. [GitHub](#)

- **Input it takes:** Python source files or directories (CLI: `bandit -r <path>`).
- **Output it gives:** Reports to stdout or file in multiple formats (plain text, JSON, XML — via formatters/plugins). [bandit.readthedocs.io+1](#)

Key features

- AST-driven checks and pluggable rule set (plugins).
- Severity & confidence classification (filterable).
- `#nosec` inline suppression, baseline/support for CI integration, and a GitHub Action.
- Extensible output formatters (JSON/XML) for CI/aggregation.

Advantages

- Low barrier: pip install bandit and run; CI-friendly outputs.
- Security-focused rule set that targets common Python vulnerabilities.
- Easy to extend with custom plugins and formatters.

Disadvantages

- Not a general linter — limited to security patterns.
- Some rules are surface-level (pattern/AST-based) and can have false positives/negatives for complex code flows.
- No dynamic or deep taint analysis (limited contextual flow analysis).

Typical use case

- Add as a pre-commit or CI step to automatically flag common security issues in Python repos and produce JSON/XML for dashboards or gating.

Notes

- Good complement to linters and SCA tools; use for quick, automated security hygiene checks before deeper/manual review.

2) ESLint — JavaScript / TypeScript (pluggable linter / static analyzer)



ESLint is the de-facto pluggable linter for ECMAScript/JavaScript (and via plugins, TypeScript). While commonly used for style, it also encodes many static checks that find problematic patterns and security-related rule plugins exist. [GitHub](#)

- **Input it takes:** JavaScript/TypeScript source files and project configuration (.eslintrc, CLI globs).
- **Output it gives:** CLI reports in many formatters (stylish, json, json-with-metadata, html, compact, etc.). Format selectable with --format/-f. [eslint.org](#)

Key features

- Pluggable rule architecture; huge ecosystem of community plugins (security, accessibility, frameworks).
- Auto-fix capability for many rules (--fix).
- Multiple built-in formatters and third-party formatters for CI/GitHub Actions.

Advantages

- Extremely configurable and extensible (rule granularity per file/dir).
- Integrates with editors and CI; can auto-fix many issues.
- Large ecosystem (rules for security, best practices, frameworks).

Disadvantages

- Large rule-sets can be noisy; requires careful config to avoid friction.
- ESLint is primarily a linter; it won't replace specialized SAST engines that do path/taint analysis.
- For TypeScript you often need @typescript-eslint tooling (extra setup).

Typical use case

- Enforce code quality and catch pattern-level bugs in JS/TS projects; integrate with CI and GitHub Actions, and apply auto-fixes on PRs.

Notes

- Use security-focused plugin packs (e.g., eslint-plugin-security) and a CI formatter that produces machine-readable output for dashboards. [GitHub+1](#)

3) SpotBugs — Java (bytecode static analyzer)



SpotBugs analyzes Java bytecode (.class/.jar) to find bug patterns (null derefs, bad API use, concurrency issues). It is the maintained successor to FindBugs and integrates with Maven/Gradle. [GitHub+1](#)

- **Input it takes:** Compiled Java artifacts (.class files, JARs) or build-integrated invocation via Maven/Gradle plugins.
- **Output it gives:** Reports in HTML, XML, text and IDE/CI consumable formats (plugins produce standard reports). [spotbugs.readthedocs.io](#)

Key features

- Bytecode analysis (so it works regardless of source layout).
- Large catalog of bug detectors (null pointer, performance, bad practice, security checks via plugins).
- Official Maven/Gradle plugins and IDE integrations.

Advantages

- Bytecode-level scanning detects issues missed by source-only checks (e.g., compiler-optimized constructs).
- Mature rule repository and integration in Java build pipelines.
- Produces developer-friendly HTML reports and CI-friendly XML.

Disadvantages

- Limited to JVM bytecode — not applicable before compilation.
- Some findings are best-effort pattern matches and may require triage (false positives).
- Less focused on deep security taint analysis compared to specialist SAST suites.

Typical use case

- Integrate as part of mvn verify/gradle check to fail builds on critical bug classes or to generate periodic quality/security reports.

Notes

- Combine with dynamic tests and dependency scanners for broader security coverage. [GitHub+1](#)

4) Cppcheck — C / C++ (static analysis focused on undefined behaviour)



Cppcheck targets C and C++ code with an emphasis on undefined behaviour, dangerous constructs, and practical false-positive reduction. It can analyze projects without full compilation data and accepts compile databases for deeper checks. [GitHub+1](#)

- **Input it takes:** C/C++ source files or project metadata (compile_commands.json, Visual Studio solutions) — `cppcheck <path>` or `--project=<file>`.
- **Output it gives:** Console output; machine formats via `--xml` / `--xml-version=2` and file redirection (commonly used with CI). [GitHub+1](#)

Key features

- Focus on undefined behavior, memory issues, style, and portability warnings.
- Can import compile databases and project files for context-aware checks.
- GUI available; supports `--enable=all` to broaden rule coverage.

Advantages

- Good for C/C++ legacy and embedded code where compilers miss logic bugs.
- Configurable to reduce false positives and supports CI-friendly XML output.
- Works with non-standard syntax common in embedded projects.

Disadvantages

- Complex preprocessor/macros may still confuse static checks — some manual config required.
- Not a full interprocedural taint engine; deep path-sensitive vulnerabilities can be missed.
- CLI options can be intricate for multi-configuration projects.

Typical use case

- Run in CI to catch undefined behavior and dangerous constructs early, or on large legacy codebases to guide modernization.

Notes

- For best results generate a `compile_commands.json` (CMake) or import IDE project files so Cppcheck can use actual preprocessor flags. cppcheck.sourceforge.io+1

5) gosec — Go (Go security checker)



gosec - Go Security Checker

A Simple Way to Secure Your Go Projects

gosec scans Go source for common security problems by inspecting the AST/SSA and supports many CI-friendly output formats (including SARIF/Sonar/JUnit). It's widely used in Go pipelines. [GitHub+1](#)

- **Input it takes:** Go source files or packages (CLI patterns, e.g., `gosec ./...`).
- **Output it gives:** Multiple formats: text, json, yaml, csv, sonarqube, JUnit XML, HTML, and golint-like output. Use `-fmt` and `-out` flags to control output. [GitHub](#)

Key features

- AST + SSA-based checks tailored for Go idioms (unsafe use, TLS issues, crypto mistakes).
- Rule configuration and ability to suppress findings (`// #nosec` comments).
- SARIF/Sonar export and GitHub Actions marketplace integration.

Advantages

- Focused on Go idioms and common Go security pitfalls.
- Flexible output formats make it easy to integrate into CI, dashboards, or upload SARIF to GitHub.
- Actively maintained and easy to run (`go install/binary`).

Disadvantages

- Rule set is pattern-based; complex interprocedural flows may be missed.
- Some false positives require triage or suppression in legacy code.
- Needs updates when language features evolve (but active community mitigates this).

Typical use case

- Add to CI (or GitHub Actions) to gate PRs for common Go security issues and export machine-readable reports for aggregation.

Notes

- To feed findings into GitHub Code Scanning use SARIF export; for fine control use a config file and `#nosec` to suppress acceptable findings. [GitHub+1](#)